Lab 3: I^{2}C You!

Hardware Abstraction

In Lab 1 & 2, you were asked to find register addresses and bit data from the reference manual.

```
#define GPIOC_ADDR 0x48000800
#define GPIOC_MODER (*(volatile uint32_t *)(GPIOC_ADDR +
0x00))
GPIOC_MODER &= ~(0b11 << (2 * 7));
GPIOC_MODER |= (0b01 << (2 * 7));
```

This is educational, but not realistic!

Hardware Abstraction

In practice, developers use hardware abstraction often provided by manufacturers.

```
/* Baremetal */
// Enable GPIOB clock
*(volatile uint32_t *)(0x40021000 + 0x4C)
|= (1 << 1); // RCC_AHB2ENR: GPIOBEN

// Set PB7 output mode
*(volatile uint32_t *)(0x48000400 + 0x00)
&= ~(0x3 << (7 * 2));
*(volatile uint32_t *)(0x48000400 + 0x00)
|= (0x1 << (7 * 2));

// Set PB7 high
*(volatile uint32_t *)(0x48000400 + 0x14)
|= (1 << 7);
```

```
/* CMSIS */
// Enable GPIOB clock
RCC->AHB2ENR |=
RCC_AHB2ENR_GPIOBEN;

// Set PB7 output mode
GPIOB->MODER &= ~(0x3 << (7 *
2));
GPIOB->MODER |= (0x1 << (7 *
2));

// Set PB7 high
GPIOB->ODR |= (1 << 7);
```

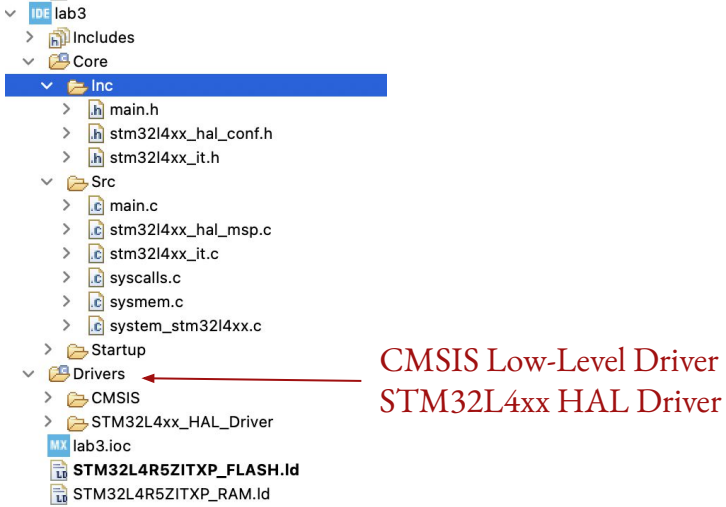
```
/* Manufacturuer HAL */
// Enable GPIOB clock
__HAL_RCC_GPIOB_CLK_ENABLE();

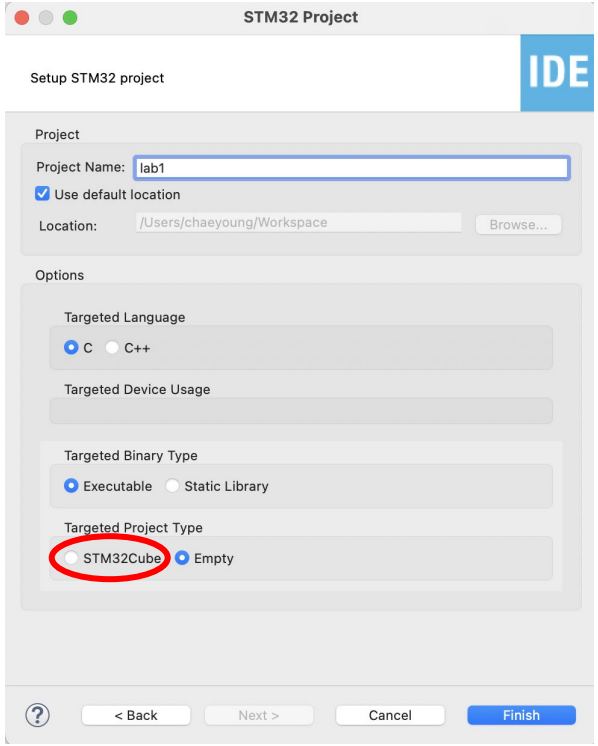
// Set PB7 output mode
GPIO_InitTypeDef GPIO_InitStruct = {0};
GPIO_InitStruct.Pin = GPIO_PIN_7;
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed =
GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

// Set PB7 high
HAL_GPIO_WritePin(GPIOB, GPIO_PIN_7,
GPIO_PIN_SET);
```

Hardware Abstraction

In this and future labs, you are allowed to use ST's HAL. To use this, when you create a project, click STM32Cube as targeted project type.

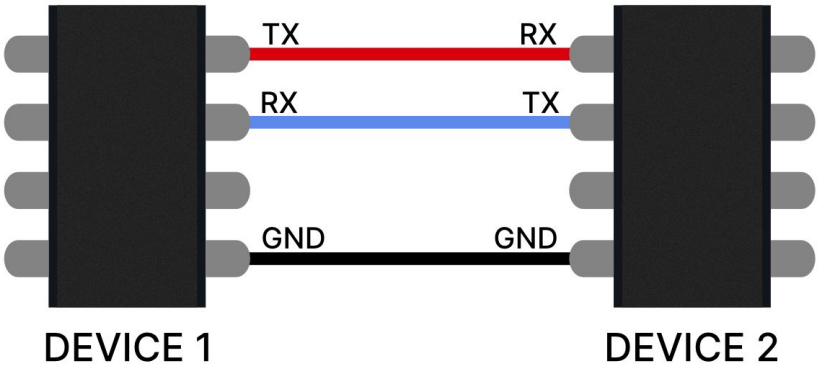




Let’s practice: UART communication

Asynchronous serial communication between devices.

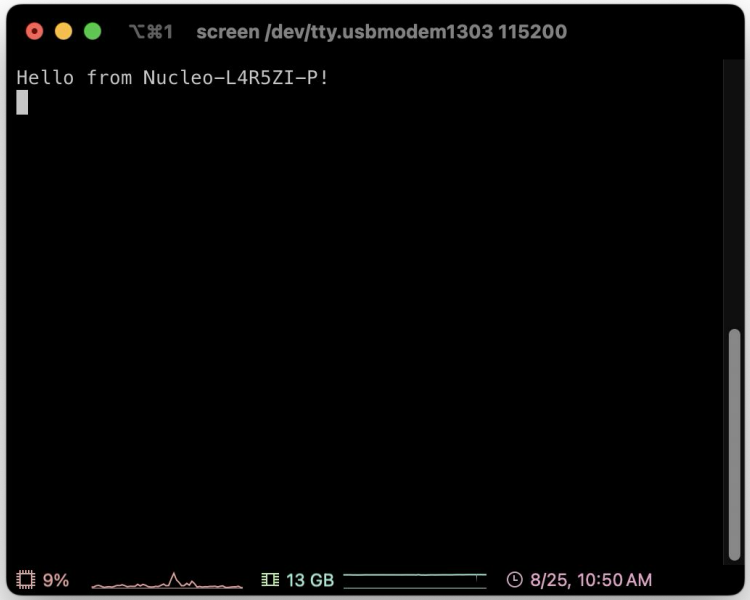
Transmits one bit at a time over a single line. There is no clock but transmitter/receiver should agree on a common baud rate (e.g. 115200 bits per second).



Let’s practice: UART communication

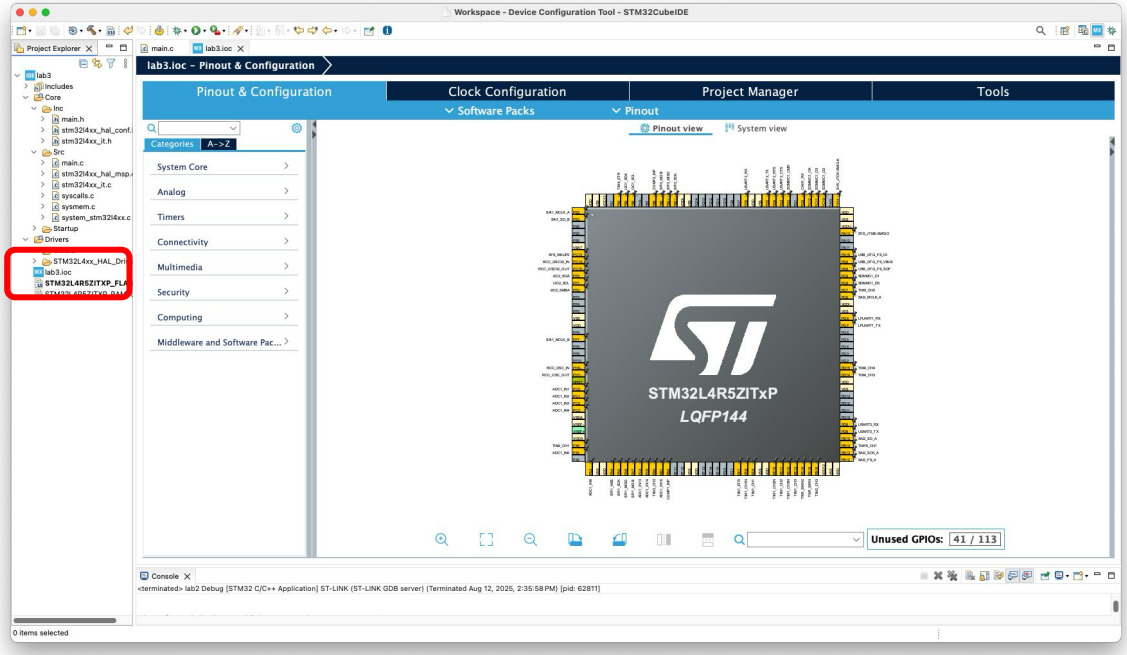
It’s useful because we can establish communication between MCU and laptop!

→ Debugging via a serial terminal (e.g., printf over UART)



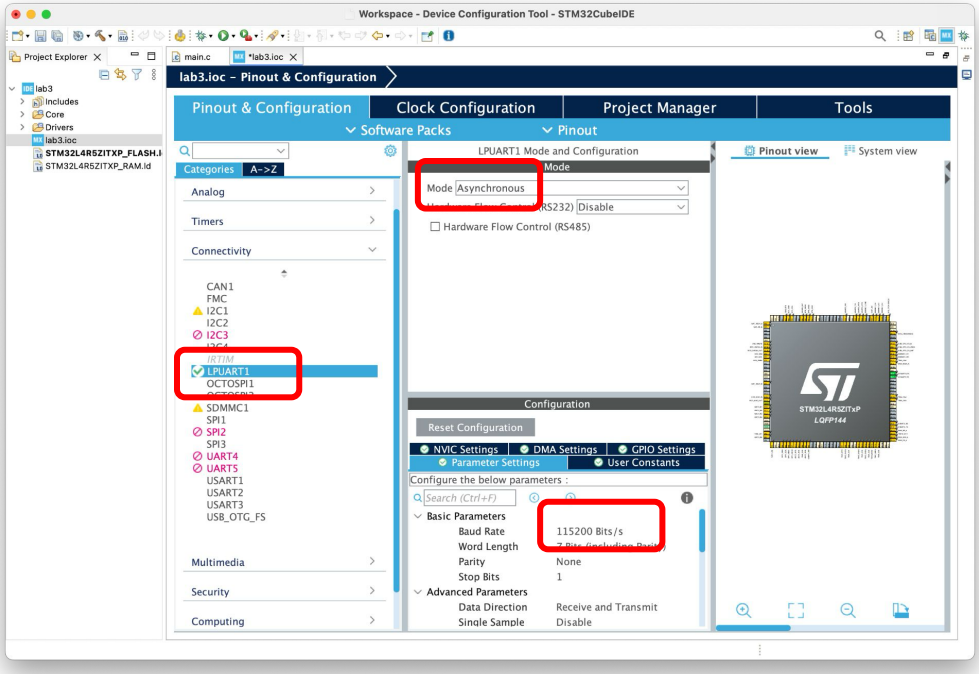
Enabling UART using HAL

You are allowed to use STM32CubeMX by double clicking the .ioc file in the project folder.



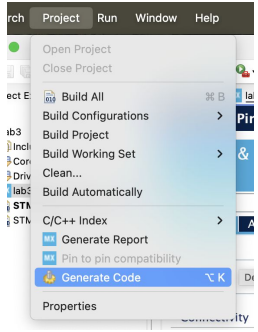
Enabling UART using HAL

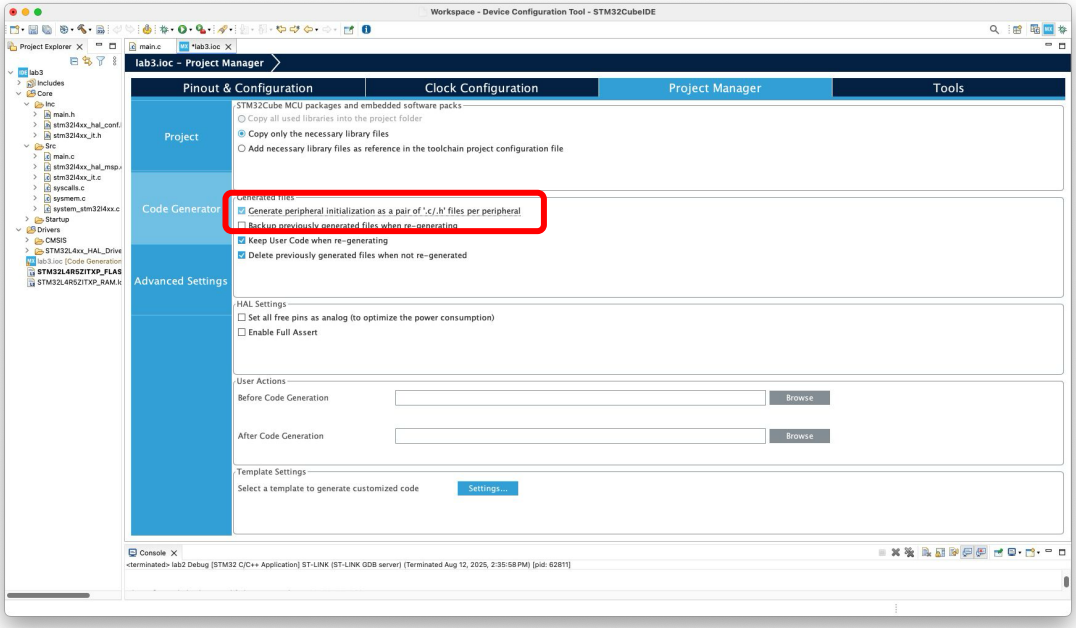
In Nucleo-L4R5ZI-P, ST-Link USB port is hard-wired to LPUART1.

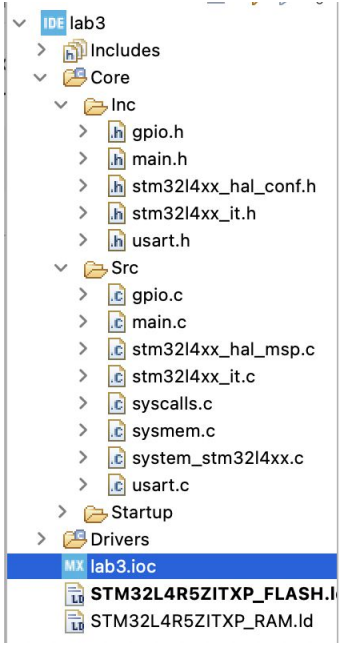


Enabling UART using HAL

Save .ioc file. If it doesn't auto-generate code, then click Project > Generate Code.







Enabling UART using HAL

```
int __io_putchar(int ch)
{
 HAL_UART_Transmit(&hlpuart1, (uint8_t *)&ch, 1, 10);
 return ch;
}
int main(void) {
    …
    MX_LPUART1_UART_Init();
    printf("\r\nHello from Nucleo-L4R5ZI-P!\r\n");
    …
}
```

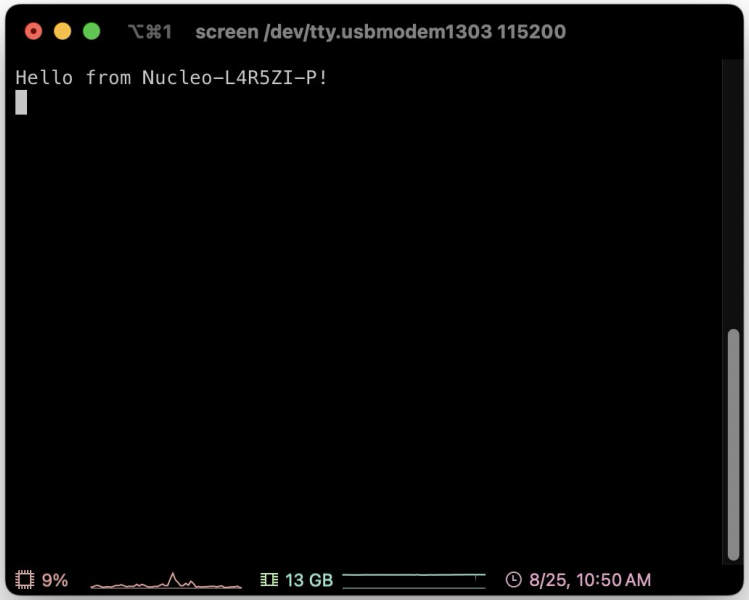
Enabling UART using HAL

In your laptop, open a serial terminal at 115200.

Windows: PuTTY

Mac/Linux: ls /dev/tty.*

screen /dev/tty.NAME 115200



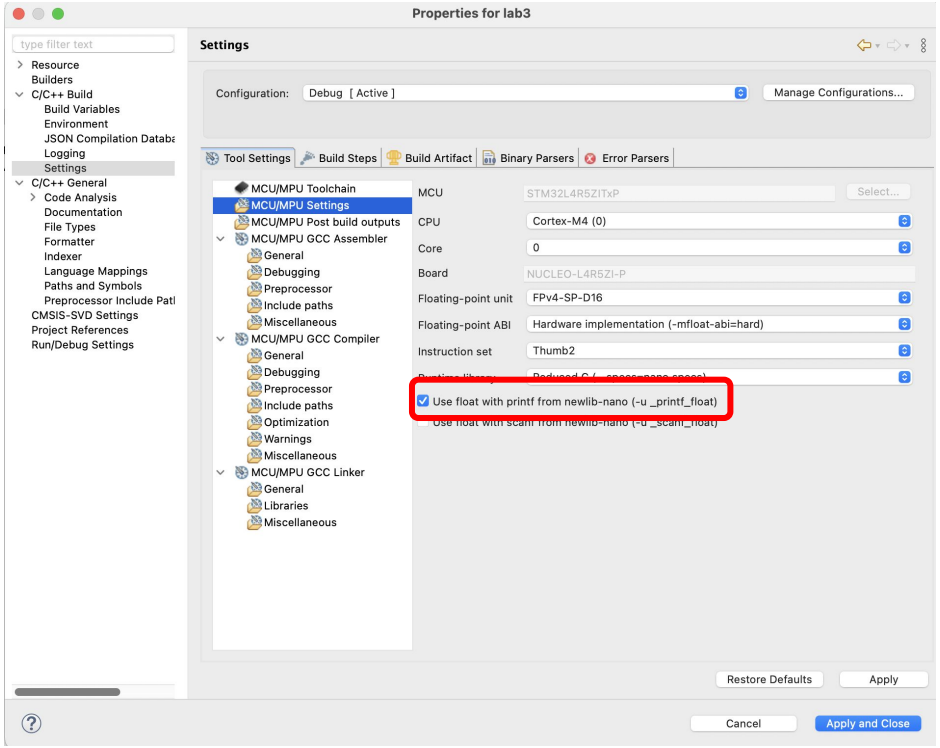
Enabling UART using HAL

Debugging tips!

- What should be the correct word length to accurately send letters? Default is set as 7-bit per character.
- What standard C libraries do you need for `strlen`, `uint8_t`?

Enabling UART using HAL

This enables floating point print.



Lab 3: I^{2}C You!

This is the first lab you are interfacing with an external peripheral!

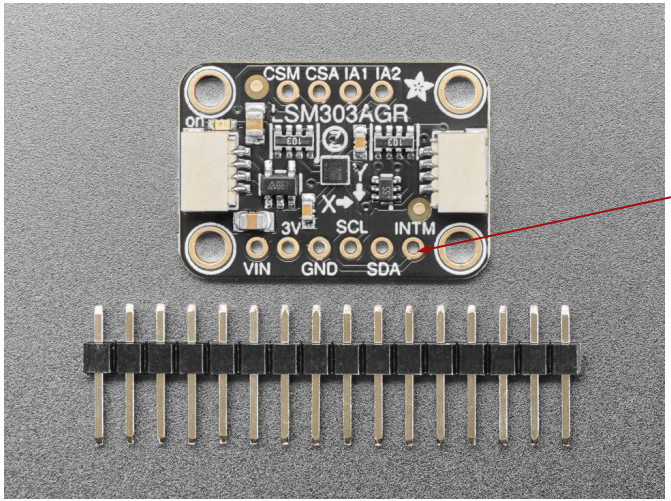
This means, you will be reading and digesting a new datasheet.

It's only 73 pages long :)

LSM303AGR

3-axis accelerometer and magnetometer.

Uses I^2C (Inter-Integrated Circuit) to communicate with MCU.



LSM303AGR

Connect the LSM303AGR module to the dev board.

LSM303AGR	Nucleo Board
VIN	?
GND	?
SCL	?
SDA	?

Table 11. ST Zio connectors pin assignments

	Pin	Pin name	Signal name	STM32 pin	Function	Remark
Left connectors						
CN8	1	NC	NC	-	-	Arduino support
	3	IOREF	IOREF		3.3 V Ref	
	5	RESET	RESET	NRST	RESET	
	7	+3.3 V	+3.3 V	-	3.3 V input/output	
	9	+5 V	+5 V		5 V output	
	11	GND	GND		ground	
	13	GND	GND			
	15	V_{IN}	V_{IN}		Power input	
	2	D43	SDMMC_D0	PC8		

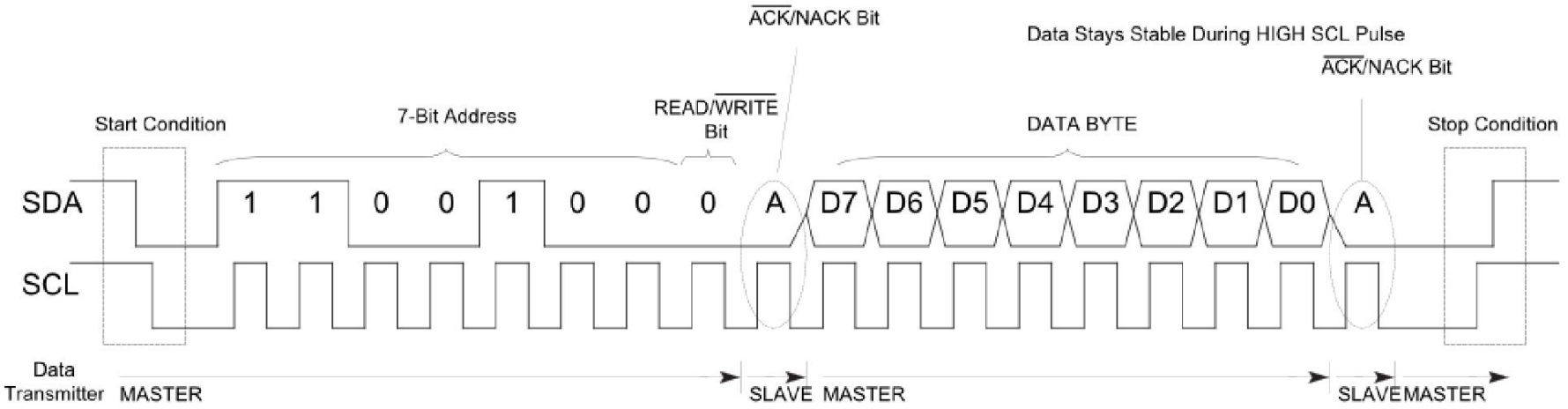
Table 11. ST Zio connectors pin assignments (continued)

	Pin	Pin name	Signal name	STM32 pin	Function	Remark
CN9	4	D52	USART_B_RX	PD6	USART_2	-
	6	D53	USART_B_TX	PD5		
	8	D54	USART_B_RTS	PD4		
	10	D55	USART_B_CTS	PD3		
	12	GND	GND	-	ground	
	14	D56	SAI_A_MCLK	PE2	SAI_1_A	
	16	D57	SAI_A_FS	PE4		
	18	D58	SAI_A_SCK	PE5		
	20	D59	SAI_A_SD	PE6		
	22	D60	SAI_B_SD	PE3	SAI_1_B	
	24	D61	SAI_B_SCK	PF8		
	26	D62	SAI_B_MCLK	PF7		
	28	D63	SAI_B_FS	PF9		
	30	D64	I/O	PG1	I/O	
Right connectors						
CN7	1	D16	I2S_A_MCK	PC6	SAI_2_A	-
	3	D17	I2S_A_SD	PB15		
	5	D18	I2S_A_CK	PB13		
	7	D19	I2S_A_WS	PB12		
	9	D20	I2S_B_WS	PA4	SAI_1_B/ SPI3(2)	
	11	D21	I2S_B_MCK	PB4		
	13	D22	I2S_B_SD/ SPI_B_MOSI	PB5		
	15	D23	I2S_B_CK/ SPI_B_SCK	PB3		
	17	D24	SPI_B_NSS	PA4		
	19	D25	SPI_B_MISO	PB4		
	2	D15	I2C_A_SCL	PB8	I2C1_SCL	Arduino support
	4	D14	I2C_A_SDA	PB9	I2C1_SDA	
	6	AREF	AREF	-	VREF+(3)	
	8	GND	GND		ground	
	10	D13	SPI_A_SCK	PA5	SPI1_SCK	
	12	D12	SPI_A_MISO	PA6	SPI1_MISO	
	14	D11	SPI_A_MOSI/ TIM_E_PWM1	PA7	SPI1_MOSI/ TIM17_CH1	

I^2C Basics

Requires SDA (data line) and SCL (clock line) for communication.

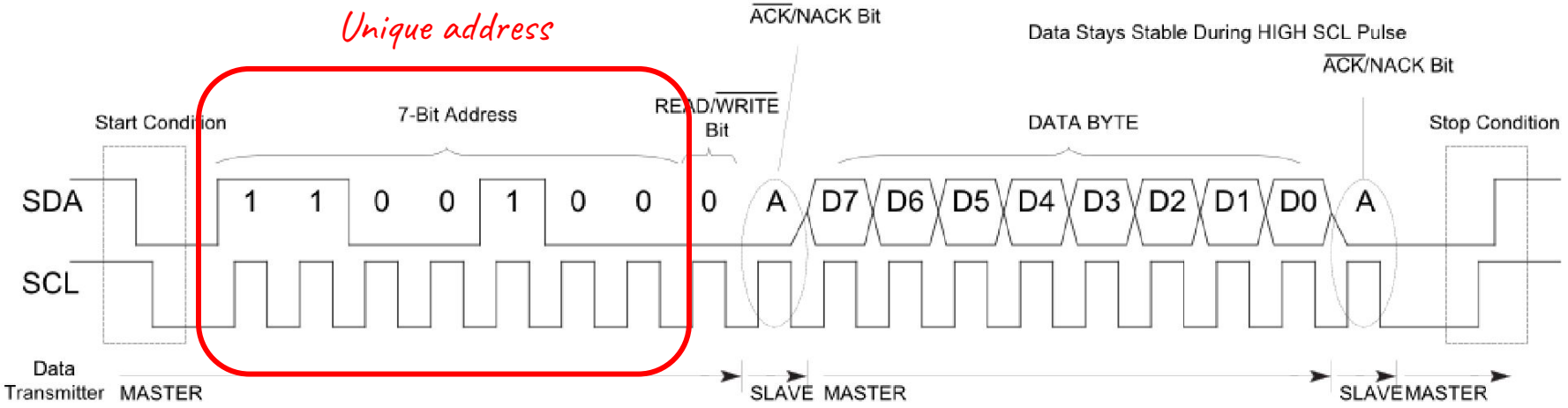
A successful write byte transmission looks like this:



Analog Devices

I²C Basics

There can be multiple controllers & peripherals listening to the same line.



I^2C Basics

What is LSM303AGR's unique I^2C address? There are separate device addresses for the accelerometer and the magnetometer. (You may represent the r/w bit as 'b').

What is the sub-address (SUB)? How is it different from I^2C address?

Table 20. Transfer when master is writing one byte to slave

Master	ST	SAD + W		SUB		DATA		SP
Slave			SAK		SAK		SAK	

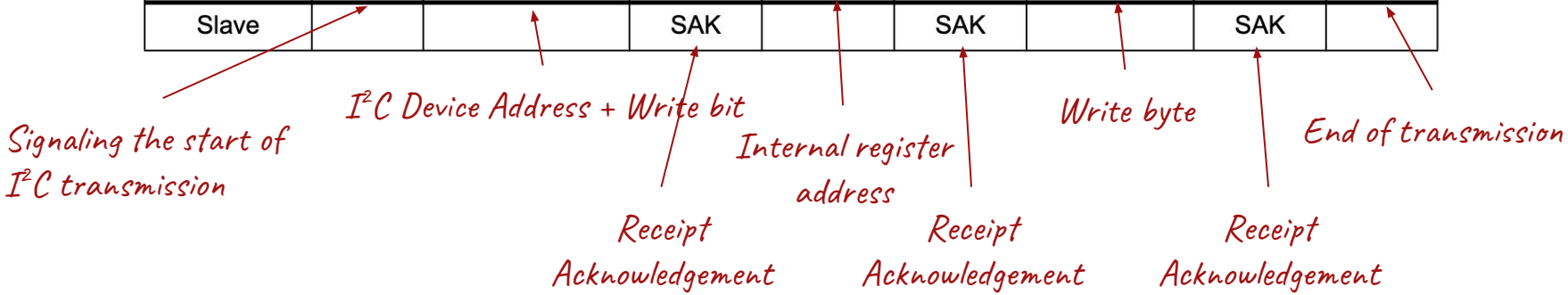
I²C Basics

What is the register address to

- Write the sensor setting (e.g. enable accelerometer, data rate)
- Read the output data

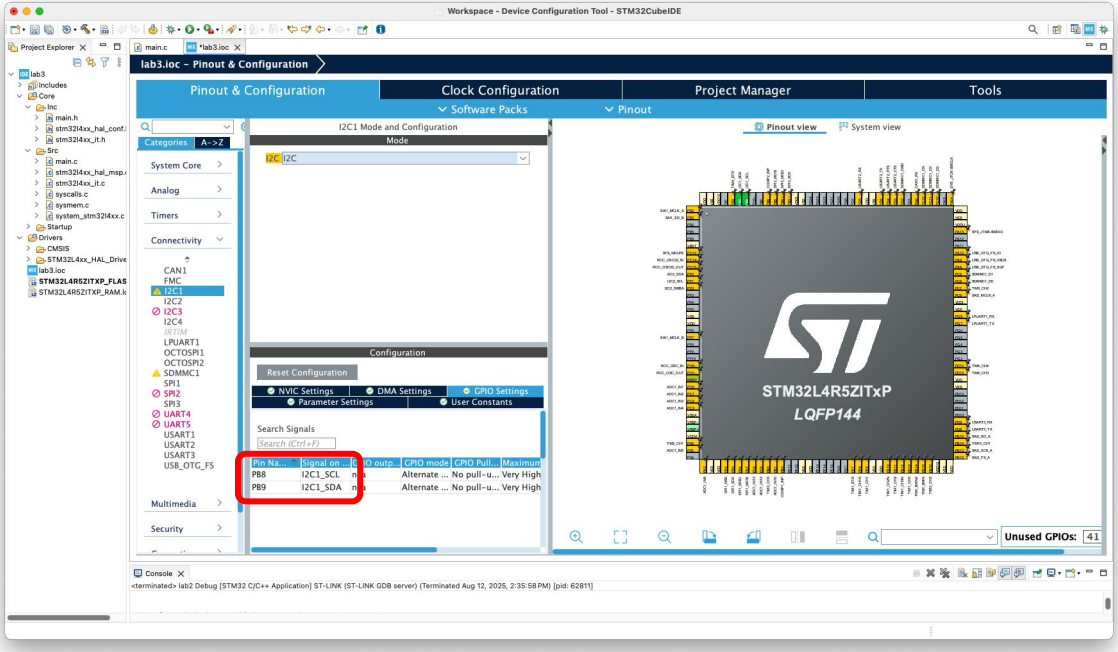
Table 20. Transfer when master is writing one byte to slave

Master	ST	SAD + W		SUB		DATA		SP
Slave			SAK		SAK		SAK	



Writing an I²C driver using HAL

Enable I2C1.



Writing an I²C driver using HAL

In main.c:

```
   */
/* USER CODE END Header */
/* Includes ------------------------------------------------------------------*/
#include "main.h"
#include "i2c.h"
#include "gpio.h"

/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */
```

```
int main(void)
{

  /* USER CODE BEGIN 1 */

  /* USER CODE END 1 */

  /* MCU Configuration--------------------------------------------------------*/

  /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
  HAL_Init();

  /* USER CODE BEGIN Init */

  /* USER CODE END Init */

  /* Configure the system clock */
  SystemClock_Config();

  /* USER CODE BEGIN SysInit */

  /* USER CODE END SysInit */

  /* Initialize all configured peripherals */
  MX_GPIO_Init();
  MX_I2C1_Init();
  /* USER CODE BEGIN 2 */

  /* USER CODE END 2 */

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
  }
  /* USER CODE END 3 */
}
```

Writing an I^2C driver using HAL

Let's check if I^2C is working by reading from these addresses.

8.4 WHO_AM_I_A (0Fh)

Table 30. WHO_AM_I register

0	0	1	1	0	0	1	1

Device identification register.

8.38 WHO_AM_I_M (4Fh)

The identification register is used to identify the device (read-only register).

0	1	0	0	0	0	0	0

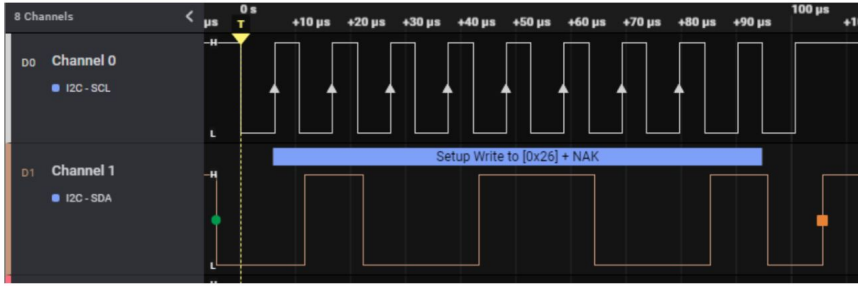
Writing an I^2C driver using HAL

```
uint8_t who_am_i;
HAL_I2C_Mem_Read(&hi2c1,
                 (0x19 << 1),              // Device address
                 0x0F,                     // WHO_AM_I register
                 I2C_MEMADD_SIZE_8BIT,     // 1-byte register address
                 &who_am_i,                // Output buffer
                 1,                        // Number of bytes
                 HAL_MAX_DELAY);           // Timeout
uart_printf(who_am_i);
```

Debugging I^2C

Sparkfun USB Logic Analyzer



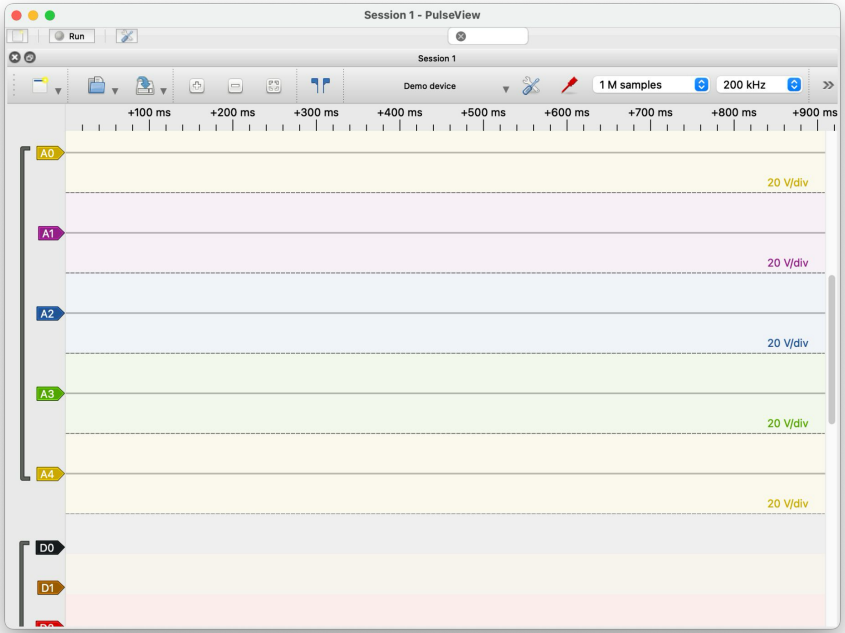


Great online tutorial:
https://learn.sparkfun.com/tutorials/using-the-usb-logic-analyzer-with-sigrok-pulseview

PulseView

Open-source signal analysis software

Note: official version for ARM Macs is unavailable, but I've found it working with PulseView 0.4.2 (.dmg is available on course website Assignments tab and Canvas Files).



You have all the tools now!



Measuring Gravity

You can measure gravity using an accelerometer!

